

Hugo Tanaka

Professor Bo Shen

CSCI 611

2026/03/13

Assignment 3

This assignment required to compare the performance of the pre-trained YOLO model and the fine tuned version to see how the model could be improved from the given dataset. The assignment also required to create the model from the scratch and see if fine tuning could also be used to improve the performance of training, validation, or testing.

The dataset for this assignment was acquired from Mapillary Traffic Sign Dataset, will be abbreviating MTSD for now on. These included photos of streets with very detailed annotations. The given dataset was already included with clear annotations; however the given annotations was written in Json format. The conversion from json format to the required text format was personally, was the hardest part of this assignment, since it took a day or so to get the whole program to correctly convert the annotations. (Image below, left: pretrained model; right: fine-tuned model)



The assignment required to create the model from scratch, which I could confidentially mention that it failed. There is no wrapping around the image that pre-trained model existed. I tried running the training multiple times to see if there is anything that I can improve upon, but I might have messed up from the base of the training itself. One thing I might have done wrong is that I put in some wrong parameters when I was training the model. According to the result.csv

files that I got it both from `base_train` and `fine_tuned` I models. One thing that might be the concern is that the both models abruptly stopped mid-training and both models got less epochs of training than I anticipated. 640 version got 4 and 1280 version only got 1; I was training on ECC-LINUX-A100, i.e. CSCIGPU, server to train these models but I think I should have watched the whole training to see what caused to stop in mid-training. I am suspecting the CPU-scheduler of the server, but I am not sure.

Although the results are unsatisfactory, I could see why having fine-tunes are really good when it comes to training. On the 4th go, the values on `base_model` of the `mAP50(B)` was 0.16937 before the training stopped, meanwhile for the `fine_tuned_model` got a equivalent score of 0.16332, on the first epoch, which means that the training was roughly 4 times faster on running with tuning and some transformations. The only downside is that the precision for the `fine_tuned` model was much lower than normal version; this is probably due to not having a great enough training to validate its results. Since it also had a much higher recall, which meant that it was detecting many wrong detection.

In conclusion, YOLO takes long time to train, even with A100 GPUs. The fine-tuning method of training the model gives more variation the set of images which would have better training results, increasing both precision and accuracy of detection, than normally training. I would love to try this again with environments with uninterrupted longer training to see more accurate and precise detection on future projects.

Results.csv (640, base model)

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
epoch	time	train/box_loss	train/cls_loss	train/dfl_loss	metrics/precision@B	metrics/recall@B	metrics/mAP50@B	metrics/mAP50-95@B	val/box_loss	val/cls_loss	val/dfl_loss	lr/pg0	lr/pg1	lr/pg2
1	508.094	2.42428	15.7747	0.84732	0.181	0.17067	0.09318	0.03992	2.43212	2.83417	0.94529	0.0006666000	0.0006666000	0.0006666000
2	1029.69	2.31718	2.91049	0.84302	0.36159	0.11731	0.11758	0.06114	2.33508	2.73834	0.93407	0.00116800	0.00116800	0.00116800
3	1552.58	2.35728	2.76067	0.82921	0.29289	0.17778	0.11785	0.05459	2.35994	2.61285	0.93467	0.00150500	0.00150500	0.00150500
4	2070.8	2.26584	2.59758	0.80924	0.35019	0.21365	0.16937	0.08359	2.25543	2.53341	0.91359	0.0012570	0.0012570	0.0012570

Results.csv (1280, fine tuned model)

[illegible]